

#중급반 4주차

그래프 이론 & DFS

<Graph Theory & DFS>

^{2p} /<그래프 이론>

^{9p} /<DFS>

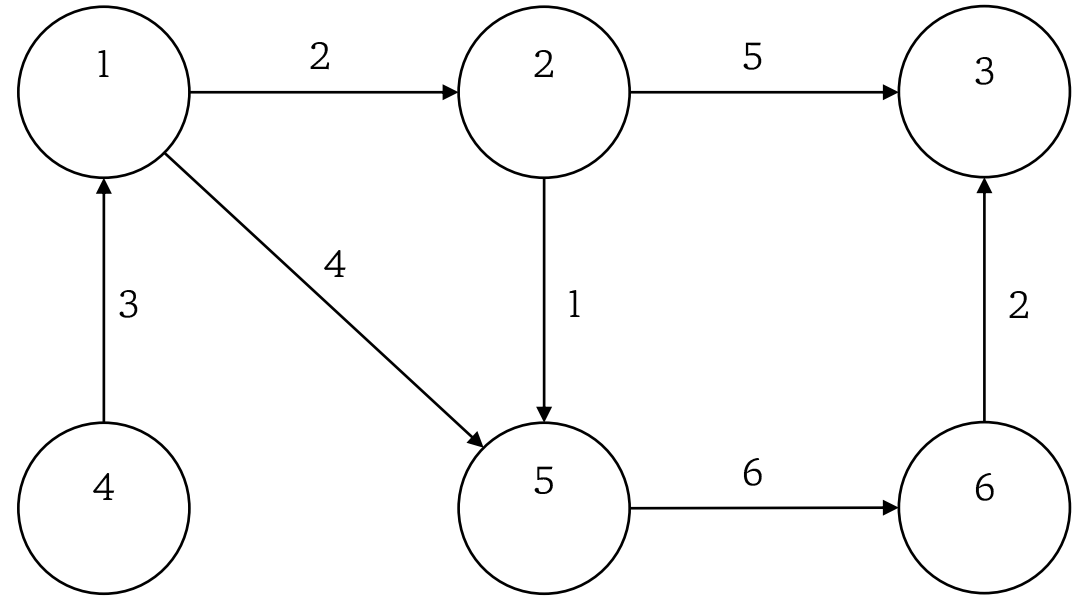
^{20p} /<연습 문제>

#그래프 이론 & DFS

<Graph Theory & DFS>

그래프란?

정점(Vertex)과 간선(Edge)으로 이루어진 구조입니다.

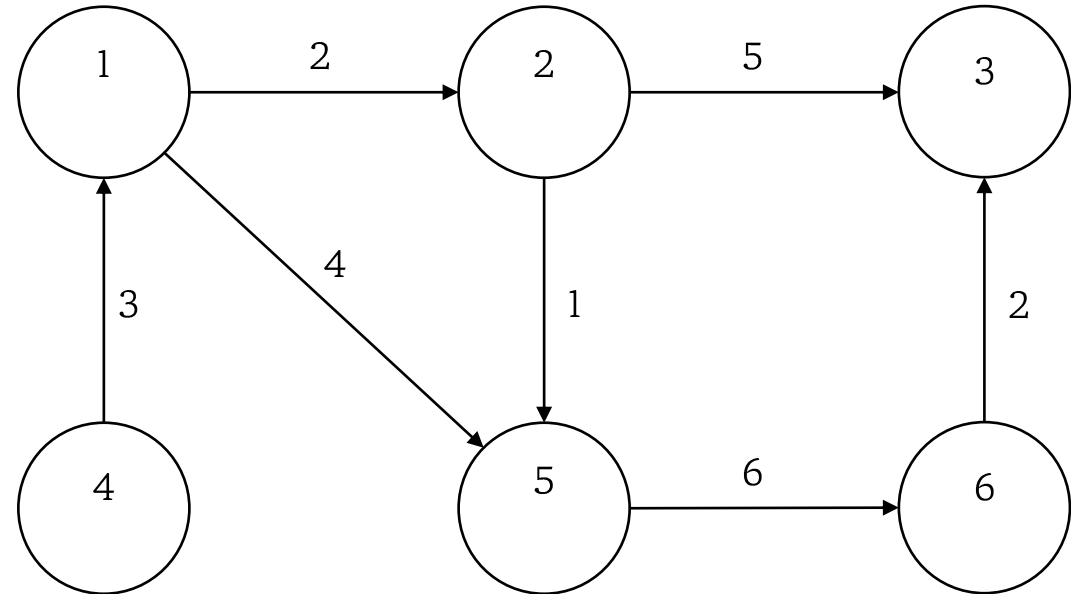


#그래프 이론 & DFS

<Graph Theory & DFS>

간선(Edge)의 여부에 따라서
그래프를 분류합니다.

1. **방향성** (양방향, 단방향)
2. **가중치**

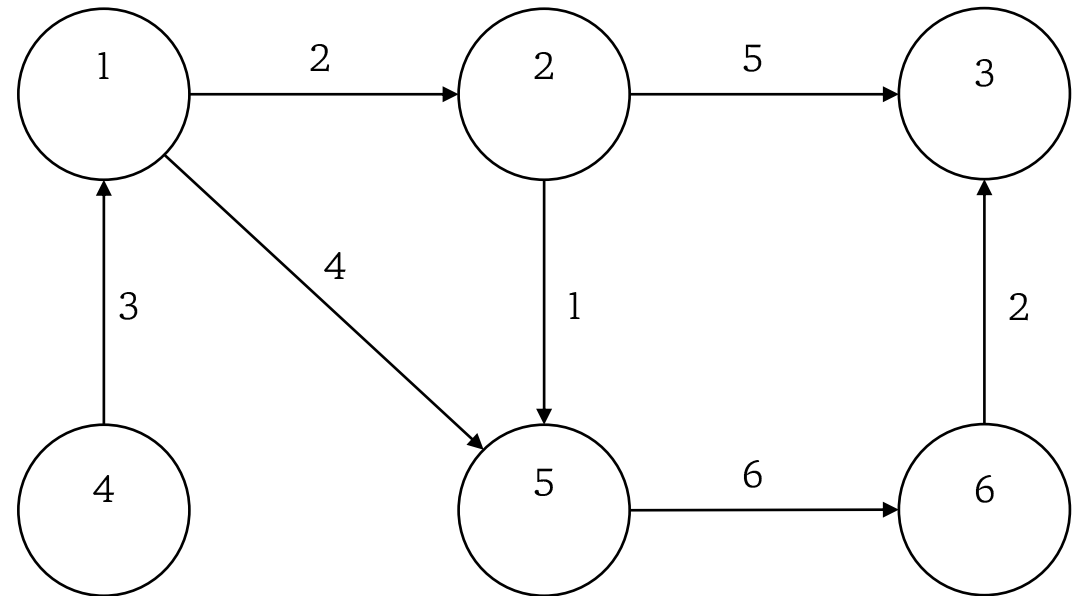


#그래프 이론 & DFS

<Graph Theory & DFS>

그래프를 저장하는 방법은 다음과 같습니다.

1. 인접 행렬
2. 인접 리스트
3. 간선 리스트



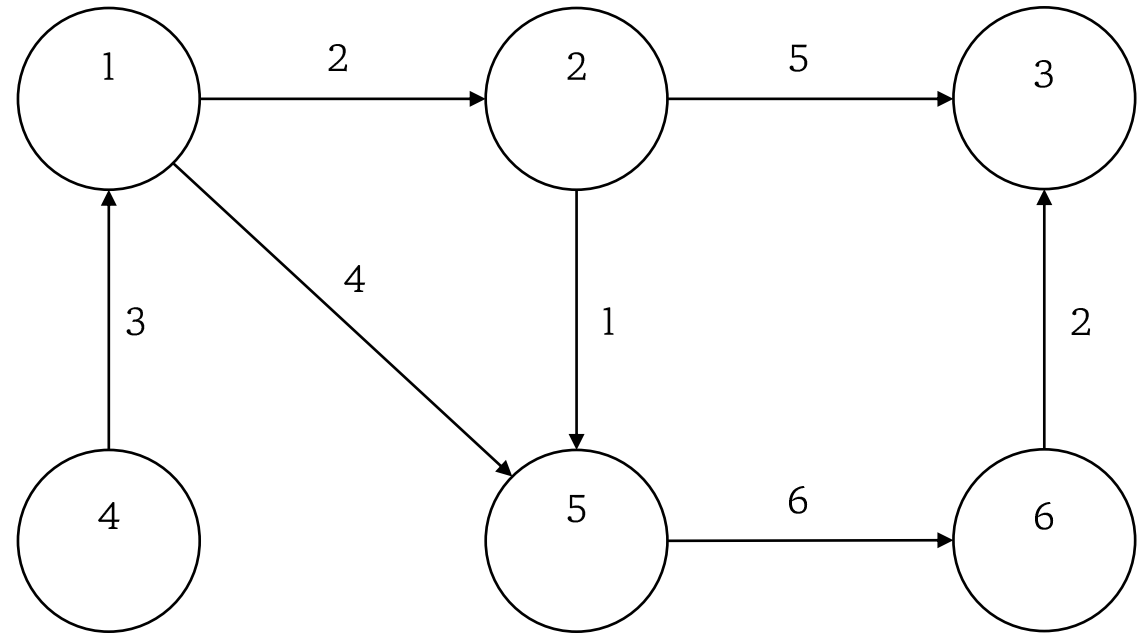
#그래프 이론 & DFS

<Graph Theory & DFS>

인접 행렬은 연결관계를
이차원 배열 형태로 나타냅니다.

$g[i][j]=w \leftrightarrow i$ 에서 j 로
가는 간선의 가중치가 w 이다.

양방향 그래프의 경우, 간선 $i \leftrightarrow j$ 가 $i \rightarrow j, j \rightarrow i$ 으로
방향성 간선2개로 취급하여 저장합니다.
가중치가 없는 경우,
 $i \rightarrow j$ 의 간선이 있으면 $a[i][j]=1$ 으로,
없으면 $a[i][j]=0$ 으로 나타냅니다.



	1	2	3	4	5	6
1	0	2	0	0	4	0
2	0	0	5	0	1	0
3	0	0	0	0	0	0
4	3	0	0	0	0	0
5	0	0	0	0	0	6
6	0	0	2	0	0	0

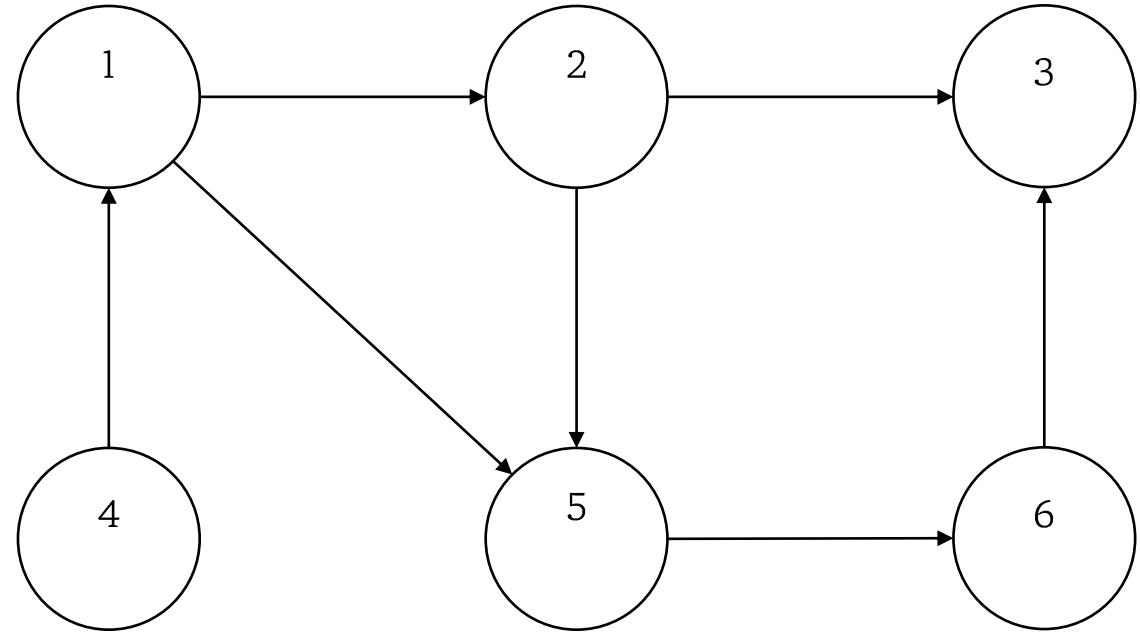
#그래프 이론 & DFS

<Graph Theory & DFS>

인접 리스트는 각 정점에 대하여 연결된 정점을 저장합니다.

$g[i]=\{j, k, l\} \leftrightarrow i$ 에서 j, k, l 로 가는 간선이 존재한다.

양방향 그래프의 경우, 간선 $i \leftrightarrow j$ 가 $i \rightarrow j, j \rightarrow i$ 으로 방향성 간선2개로 취급하여 저장합니다.
가중치가 있는 경우, $i \rightarrow j$ 의 간선이 가중치 w 로 있으면 $a[i]=\{j, w, \dots\}$ 와 같은 형태로 저장합니다.



$g[1]=\{2,5\}$
 $g[2]=\{3,5\}$
 $g[3]=\{\}$
 $g[4]=\{1\}$
 $g[5]=\{6\}$
 $g[6]=\{3\}$

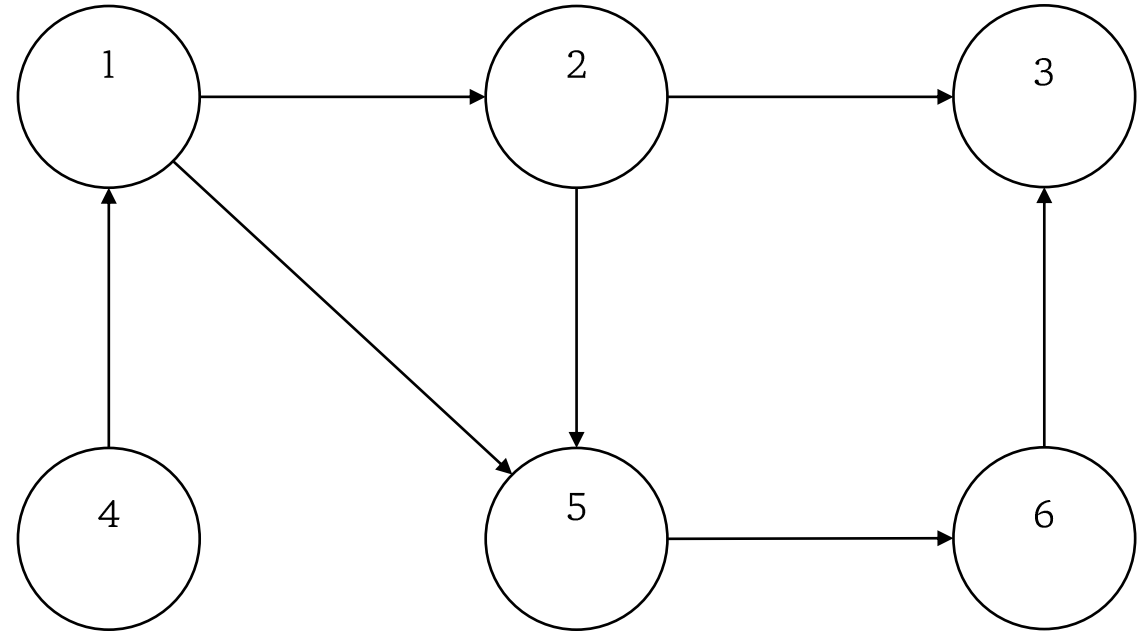
#그래프 이론 & DFS

<Graph Theory & DFS>

간선 리스트는 모든 간선을
리스트에 저장합니다.

$g = \{\{i, j\}, \dots\} \leftrightarrow$
 $i \rightarrow j$ 로 가는 간선이 존재한다.

가중치가 있는 경우, $i \rightarrow j$ 의 간선이 가중치 w 로 있으면
 $g = \{\{i, j, w\}, \dots\}$ 와 같은 형태로 저장합니다.



$g = \{\{1, 2\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{4, 1\}, \{5, 6\}, \{6, 3\}\}$

#그래프 이론 & DFS

<Graph Theory & DFS>

보통 인접 리스트로 그래프를 저장하며
구현 코드는 다음과 같습니다.

```
#include<bits/stdc++.h>

using namespace std;
vector<int> adjacent[123456];

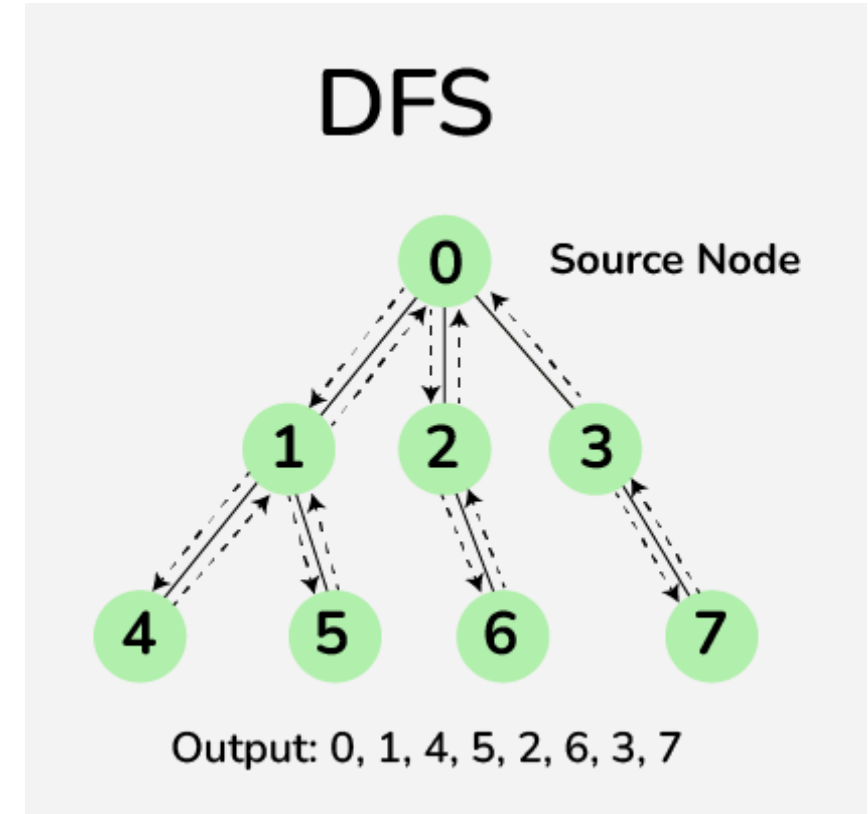
int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < m; i++){
        int x, y;
        cin >> x >> y;
        adjacent[x].push_back(y);
    }
}
```


#그래프 이론 & DFS

<Graph Theory & DFS>

DFS란 Depth First Search(깊이 우선 탐색)의 약자로 그래프를 순회하는 방법 중 하나입니다.

한 정점에서 다른 정점으로 이동하며 더 이상 이동이 불가능 할 때 멈추게 되는 알고리즘입니다.



<https://www.geeksforgeeks.org/difference-between-bfs-and-dfs/>

#그래프 이론 & DFS

<Graph Theory & DFS>

3 바이러스 (BOJ #2606)

<문제 설명>

- 컴퓨터가 n 개 있으며 네트워크 상 연결 m 개가 존재한다.
- 1번 컴퓨터가 바이러스에 감염이 되었다.
- 네트워크 상 연결을 통하여 바이러스가 퍼져간다.
- 1번 컴퓨터를 통하여 감염되는 컴퓨터의 수를 출력한다.

<제약조건>

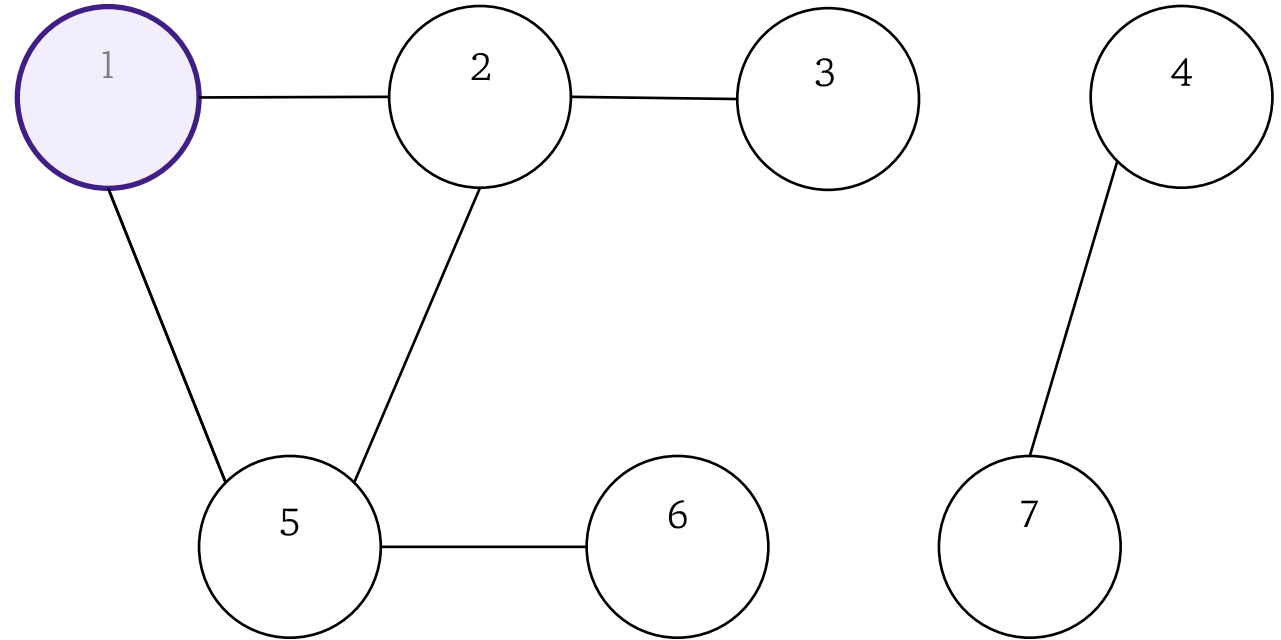
$$1 \leq n \leq 100$$

#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

1에서 시작합니다.

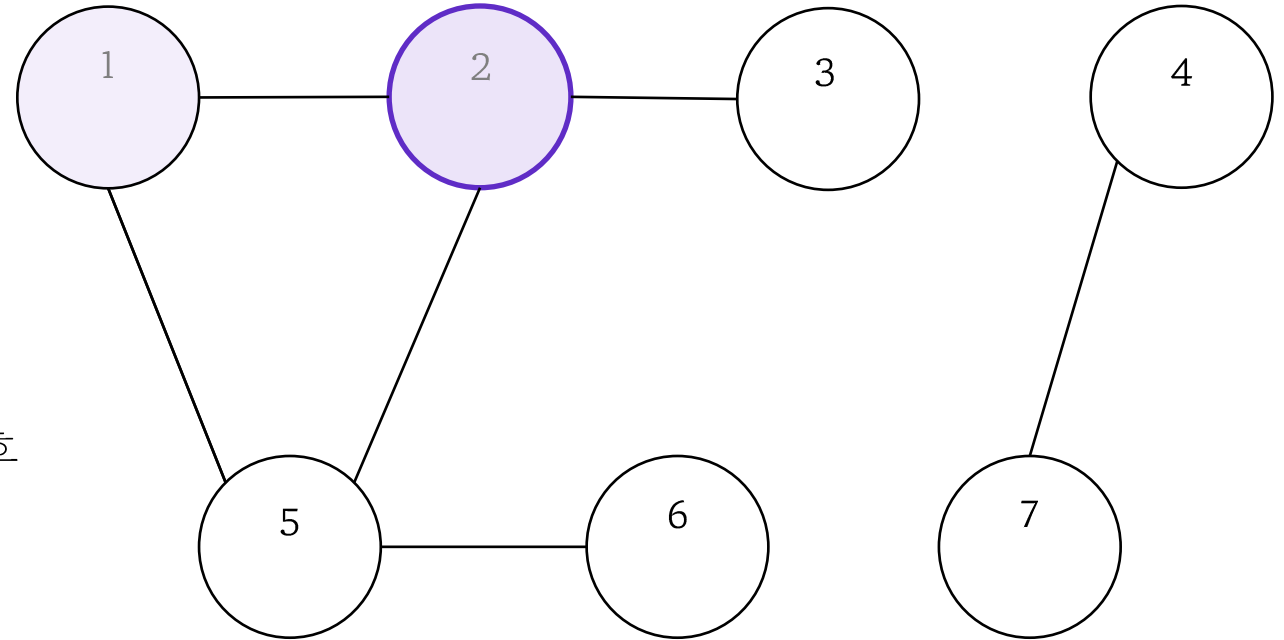


#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

1에서 2를 탐색하게 됩니다. 이후 탐색이 종료 후
5를 탐색하게 됩니다.



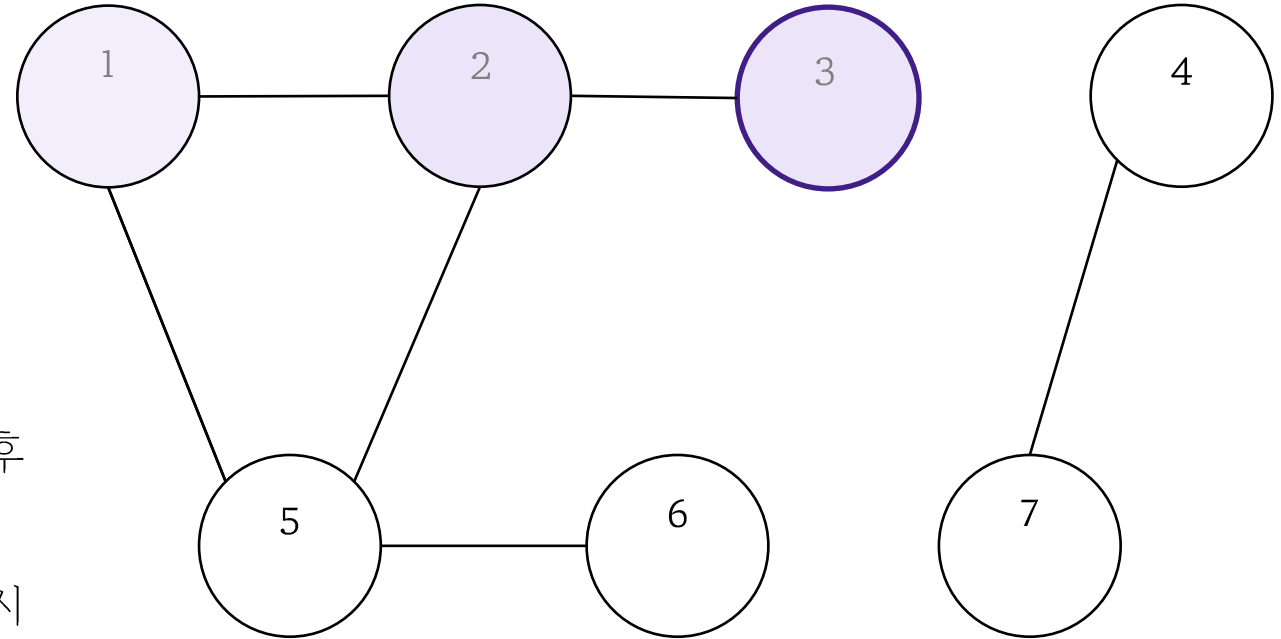
#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

2에서 3을 탐색하게 됩니다. 이후 탐색이 종료 후
5를 탐색하게 됩니다.

3에서 더 이상 탐색할 수 없기에 2로 돌아가 다시
탐색하게 됩니다.

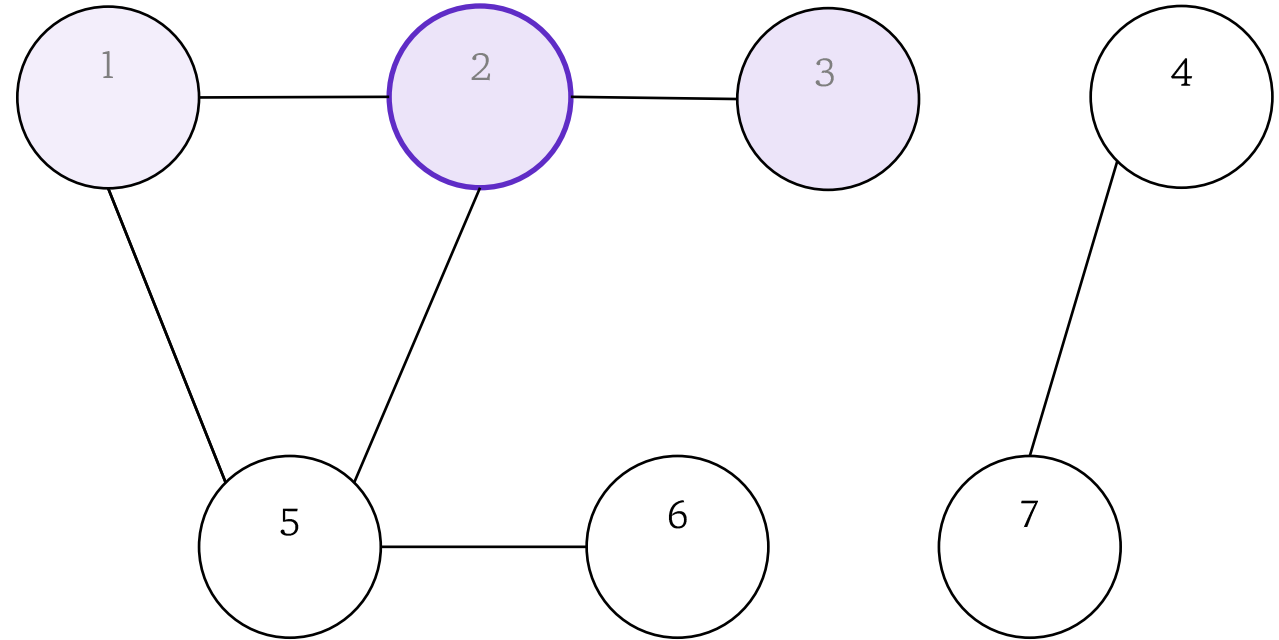


#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

다시 2로 돌아가게 됩니다.



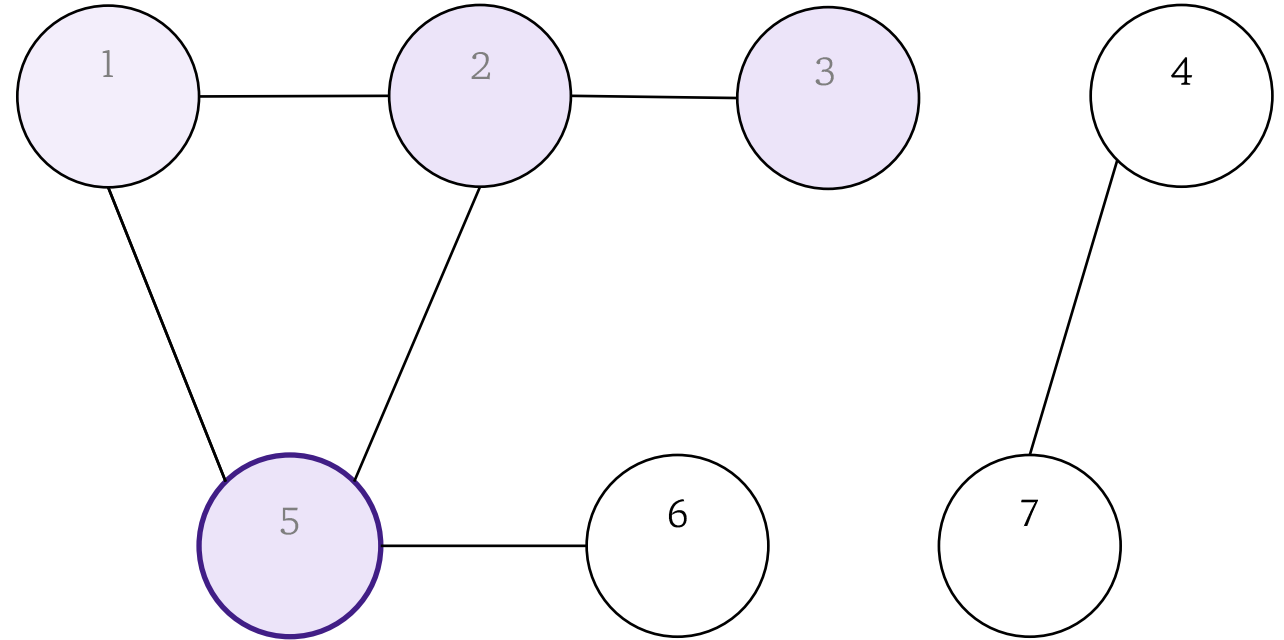
#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

2에서 5로 이동하여 탐색하게 됩니다.

1은 이미 탐색하였기에 6을 탐색하게 됩니다.



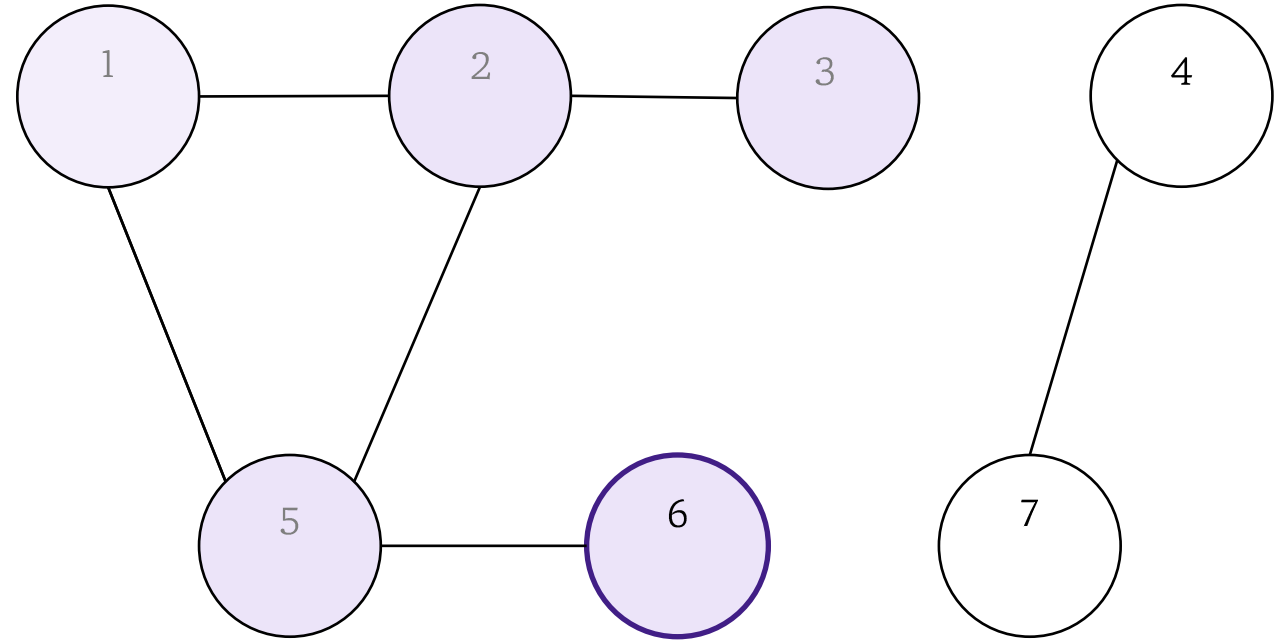
#그래프 이론 & DFS

<Graph Theory & DFS>

예제1을 그려보면 오른쪽과 같이 나오며
DFS를 활용하면
다음의 순서로 풀 수 있습니다.

6에서 더 이상 탐색 할 수 있는 정점이 없기에
5로 돌아가게 됩니다.

5,2,1로 차례로 돌아가게 되어 종료하게 됩니다.



#그래프 이론 & DFS

<Graph Theory & DFS>

이제 위의 과정을 코드로 구현하면 다음과 같습니다.

```
int ans = 0;
vector<int> adjacent[111];
vector<bool> visited(111, false);

void dfs(int cur){
    visited[cur] = true;
    ans++;
    for(int i = 0; i < adjacent[cur].size(); i++){
        int ch = adjacent[cur][i];
        if(!visited[ch])
            dfs(ch);
    }
}

int main(){
    ...
    dfs(1);
    ...
}
```

#그래프 이론 & DFS

<Graph Theory & DFS>

현재 방문한 정점에 방문표시를 합니다.

```
int ans = 0;
vector<int> adjacent[111];
vector<bool> visited(111, false);

void dfs(int cur){
    visited[cur] = true;
    ans++;
    for(int i = 0; i < adjacent[cur].size(); i++){
        int ch = adjacent[cur][i];
        if(!visited[ch])
            dfs(ch);
    }
}

int main(){
    ...
    dfs(1);
    ...
}
```

#그래프 이론 & DFS

<Graph Theory & DFS>

방문 할 수 있는 정점 중 방문하지 않은 것에
대하여 dfs를 반복합니다.

```
int ans = 0;
vector<int> adjacent[111];
vector<bool> visited(111, false);

void dfs(int cur){
    visited[cur] = true;
    ans++;
    for(int i = 0; i < adjacent[cur].size(); i++){
        int ch = adjacent[cur][i];
        if(!visited[ch])
            dfs(ch);
    }
}
```

```
int main(){
    ...
    dfs(1);
    ...
}
```

for(int i = 0; i < adjacent[cur]; i++) 를
for(auto ch: adjacent[cur]) 로 대신 써도 됩니다.

#그래프 이론 & DFS

<Graph Theory & DFS>

그래프 외에서도 DFS를 활용할 수 있습니다.

옆은 배열에서 DFS를 사용하는 예시입니다.

일반적으로 DFS는

1. 현재 상태를 처리했다고 표시
 2. 될 수 있는 다음 상태 중에서 처리하지 않은 것에 대하여 반복
- 의 구조를 가집니다.

```
int b[66][66];
bool visited[66][66];
int n;

void dfs(int x, int y){
    visited[x][y] = true;
    int d = b[x][y];
    if(x + d < n && !visited[x + d][y])
        dfs(x + d, y);
    if(y + d < n && !visited[x][y + d])
        dfs(x, y + d);
}
```

#필수 문제

3 바이러스 (BOJ #2606)

위에서 설명한 문제입니다.

3 도시와 비트코인 (BOJ #31575)

배열 위에서 DFS를 해 봅시다.

#연습 문제 도전

2 촌수계산 (BOJ #2644)

DFS를 이용하여 두 정점 사이의 거리를 구해봐요!

2 알고리즘 수업 – 깊이 우선 탐색 6 (BOJ #24484)

각 정점의 깊이도 저장해 봅시다.

2 트리의 부모 찾기 (BOJ #11725)

트리는 특수한 형태의 그래프입니다.

2 알고리즘 수업 – 깊이 우선 탐색 1 (BOJ #24479)

DFS를 사용하면 어떤 순서로 정점을 탐색할까요?

2 연결 요소의 개수 (BOJ #11724)

연결 요소(Connected Component)에 대해 알아 봅시다.

1 단지번호붙이기 (BOJ #2667)

배열에서의 DFS + 연결 요소의 크기 구하기

#연습 문제 도전

1 이웃 마을 (BOJ #34846)

2025 HCPC Beginner Div. B번

5 트리 (BOJ #1068)

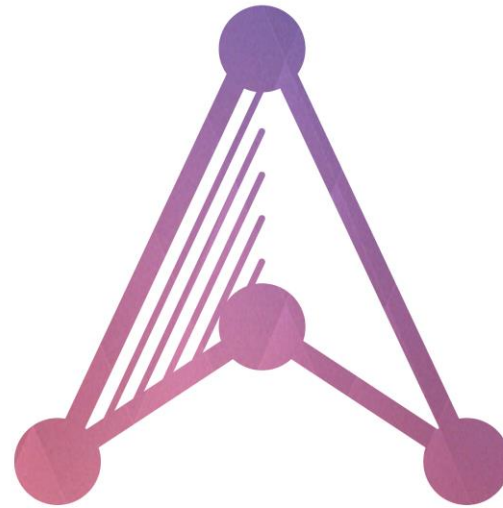
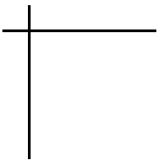
트리는 특수한 형태의 그래프입니다.

5 적록색약 (BOJ #10026)

조건이 조금 많은 그래프 탐색

4 알파벳 (BOJ #1987)

나중에 다시 풀어보면 더 좋은 문제



A L O H A
The algorithm club.

